

Dynamic Optical Pluggable Allocation and LLM-aware Job Scheduling for Distributed AI training in Regional Multi-Cluster Data Centers

Varsha Lohani⁽¹⁾, Raul Muñoz⁽¹⁾, Ramon Casellas⁽¹⁾

⁽¹⁾ Centre Tecnològic de Telecomunicacions de Catalunya (CTTC-CERCA), Spain, vlohani@cttc.es

Abstract

We propose an LLM-aware dynamic optical pluggable allocation for regional multi-cluster data centers, jointly optimizing DCI capacity and job queuing to support heterogeneous distributed AI training workloads, improving utilization, blocking probability, and training efficiency significantly. ©2026 The Author(s)

Introduction

The rapid scaling of large language models (LLMs) has driven the adoption of multiple complementary distributed workload strategies for AI training, each designed to address a different hardware bottleneck [1]. Data parallelism (DP) [2] serves as the baseline approach, replicating model parameters across GPUs and synchronizing gradients through all-reduce operations. However, this approach becomes insufficient once the model size exceeds the memory capacity of a single device. To overcome this limitation, tensor parallelism (TP) splits layer computations across GPUs with full-mesh communication [3], while pipeline parallelism (PP) partitions the model into sequential depth-wise stages with chain-like dependencies between adjacent servers [4]. In practice, large-scale dense-model training often combines TP and PP, using high-bandwidth communication within a data center for TP and lower speed communication between data centers for PP [5]. As model sizes have continued to grow, mixture-of-experts (MoE) models have further improved scalability by decoupling parameter count from per-token computation, routing each token to only a sparse subset of experts (multiple sub-models) [6–8]. This sparsity reduces computation cost while introducing irregular all-to-all communication patterns. Additionally, checkpointing [9] remains an unavoidable component of large-scale distributed training, as training systems operating across thousands of GPUs must periodically replicate model and optimizer states to ensure resilience to hardware failures.

Consequently, as AI models expand to billions of parameters [10], severe power and space limitations are bottlenecking single-facility operations. While modern infrastructure relies on scaling up compute within nodes and scaling out across local fabrics, these physical constraints force architectures to scale across multiple distributed DCs [11]. United by high-capacity data center interconnect (DCI) networks, these networks span campus (10–20 km) and regional (up to 100 km) distances [12, 13]. Regional multi-cluster DC offers a prac-

tical balance of low latency, high bandwidth, and geographic reach, making it well-suited for distributed AI job training without incurring the penalties of long-haul links. To efficiently support such deployments, we consider a full-mesh regional DCI network built on direct, high-capacity point-to-point DWDM optical links between every pair of DCs. By utilizing high-capacity DCI and optical circuit switching (OCS)-based Data center Network (DCN) within a DC, these distributed DC nodes can perform synchronized training and inference as a single virtual DC. This fully connected architecture accommodates both the sequential communication patterns of PP and many-to-many traffic generated by TP, DP, MoE, and checkpointing. However, at each DC node, statically provisioning of expensive optical pluggable transceivers to a DCI link between DC nodes results in significant under-utilization due to the highly dynamic nature of AI training traffic. During PP, communication follows a linear chain (e.g., $DC_1 \rightarrow DC_2 \rightarrow DC_3 \dots$), leaving much of the network idle. Conversely, phases such as DP, TP, MoE, and checkpointing require high-capacity, full-mesh connectivity. To resolve this inefficiency, we dynamically allocate pluggables from the shared pool according to the instantaneous topological requirements of the workload. By allowing high-demand links to have idle transceivers, the network reconfigures capacity on demand, thereby maximizing overall hardware utilization.

In this work, we propose a LLM-Aware dynamic optical pluggable allocation for distributed AI training across regional multi-cluster DCs. To capture diverse workload requirements, we model three distinct types of AI training jobs: one generating deterministic network traffic, and two exhibiting stochastic traffic patterns. To evaluate the efficiency of the proposed work, we investigate two provisioning strategies. 1. *Without Job Queuing*, immediately blocks a job if the required resources are unavailable. 2. *With Job Queuing*, as AI training jobs, unlike real-time inference workloads, can

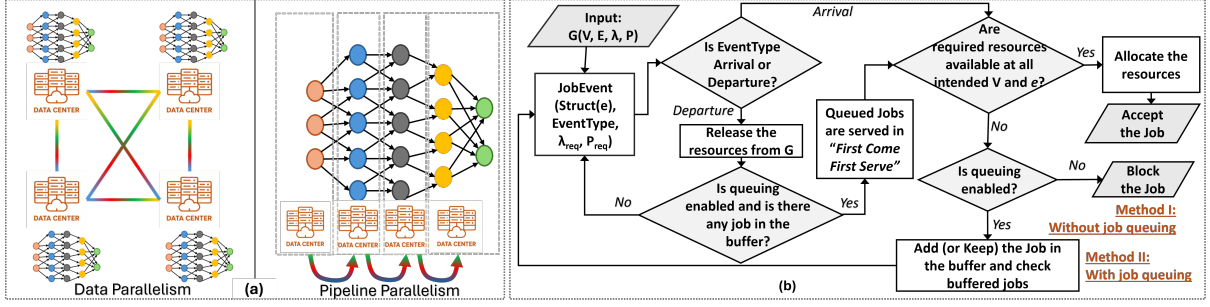


Fig. 1: (a) Two network structures for distributed AI training (b) Flowchart for optical resource allocation for AI jobs.

tolerate initial scheduling delays of a few hours [14], so instead of blocking, jobs are queued in an unbounded buffer. Finally, we compare the performance of both queuing strategies alongside static versus dynamic transceiver allocation for different AI workloads.

Proposed Methodology

In this work, we classify AI training workloads into three job types: A (TP, DP), B (PP), and C (MoE and checkpointing) based on their traffic patterns, topologies, and bandwidth requirements. As a representative scenario, we consider training a 70B-parameter model over 115 days across five DCs, each hosting 512 GPUs (2560 GPUs in total) [15]. Based on standard transformer scaling characteristics and tensor-parallel communication patterns, the resulting communication demand is on the order of tens of Tbps and is distributed across DC pairs according to the workload structure. Job Type A is deterministic, long-lived traffic (e.g., TP), characterized by tightly coupled synchronization across all participating DCs. This results in a full-mesh communication pattern (Fig. 1(a)) across DCs with ultra-high bandwidth demand (on the order of tens of Tbps per DC pair) sustained over long durations (e.g., days to weeks). Job Type B represents PP, which follows a linear-chain topology across DCs (Fig. 1(a)) where communication is localized to adjacent nodes and exhibits stochastic arrivals with moderate bandwidth demand (substantially lower than Job Type A) over durations of several hours. Job Type C captures spiky operations such as MoE and model checkpointing. These workloads require full-mesh connectivity and arrive stochastically and stay for short durations (minutes).

In Fig. 1(b), we formalize the optical physical layer as a graph $G(V, E, \lambda, P)$, where V represents the DC and E denotes the fiber links. Each link $e \in E$ supports a set of wavelengths λ , and each node is equipped with a set of pluggable transceivers P . Each incoming job is formally defined as a tuple: $\text{Job}(\text{Struct}(e), \text{EventType}, \lambda_{\text{req}}, P_{\text{req}})$. Here, $\text{Struct}(e)$ specifies the required network topology for the AI training parallelism (e.g., full-mesh or linear-chain) where $e \in E$, EventType indicates whether the event is a job arrival or departure, and λ_{req} and P_{req} denote the required number of wave-

lengths and pluggable transceivers, respectively.

We consider a DWDM-based DCI in which connectivity is established via point-to-point optical links. Upon a job arrival event, resource availability is assessed based on the job's $\text{Struct}(e)$. For full-mesh topologies (e.g., DP, TP, MoE), the required wavelengths λ_{req} and pluggables P_{req} must be simultaneously available across all nodes and links. Whereas, for linear-chain topologies (e.g., PP), required optical resources are checked sequentially along the communication path. This requires that every traversed link carries the desired wavelengths and that every intermediate node possesses sufficient pluggable transceivers. We consider two provisioning methods for pluggable transceivers. In the static allocation model, each DC is provisioned with a set of pluggables, and a dedicated subset is provisioned to each link. In contrast, in the dynamic allocation case, pluggables are maintained as a shared pool at each DC and are allocated to DC pairs based on workload requirements. In this work, we consider two job allocation methods as outlined in Fig. 1(b) and are further modeled for static and dynamic transceiver allocation.

1. Without Job Queuing: In this approach, an incoming job is accepted only if all required optical resources are available across the requested DC node pairs. Otherwise, the job is blocked.

2. With Job Queuing: In this approach, a job is accepted only if all required optical resources are available across the requested DC node pairs. This approach introduces an unbounded buffer to queue deferred jobs instead of blocking them immediately. When a departure event occurs, the allocated optical resources are released, triggering the scheduler to re-evaluate pending jobs. The buffer operates using a First Come First Serve (FCFS) policy. However, to mitigate Head-of-Line blocking, the scheduler employs strict backfilling. If the job cannot be provisioned due to the lack of optical resources, it is bypassed, allowing subsequent jobs to be considered for allocation.

Performance Evaluation

To evaluate our methodology, we simulate a 5-node full-mesh DCI network [16]. Each 50 km physical link consists of two parallel fibers operating over the Super-C and Super-L bands, providing

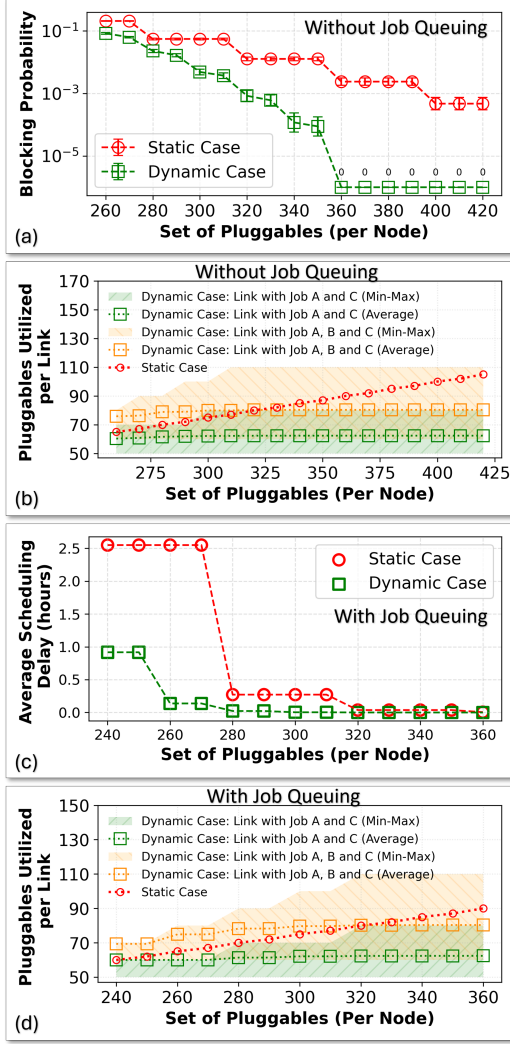


Fig. 2: Performance evaluation of static and dynamic pluggable allocation without and with job queuing.

a combined bandwidth of 12 THz (6 THz per band). Since each point-to-point connection spans only 50 km, here we use 800 ZR coherent pluggable transceivers [17] with 150 GHz channel spacing and a spectral efficiency of 5.33 bit/s/Hz; each fiber supports 80 wavelength channels, yielding 160 total wavelengths per node pair. Network performance is assessed using two primary metrics: Blocking Probability and Scheduling Delay. The latter is defined as the time a requested job waits in a buffer until sufficient pluggables become available, evaluated across varying transceiver pool sizes. A simulation time of 48-hours, with performance metrics averaged across 1000 independent iterations and blocking probability reported with a 95% confidence interval. To capture realistic AI training workloads, we model three job types with distinct traffic characteristics and resource demands. Job A is a deterministic traffic, representing long-running TP with a full-mesh communication pattern, requiring 40 Tbps of bandwidth ($\lambda_{\text{req}} = 50$) and lasting for several days. Jobs B and C introduce a structured stochastic traffic pattern. Job B, such as PP, follows a linear-chain topology with moderate

bandwidth demand of 8 Tbps ($\lambda_{\text{req}} = 10$), arrival rate of 1/6, and durations ranging from 2 to 6 hours. Job C, representing MoE and checkpointing traffic, exhibits short-lived but full-mesh communication, also requiring 8 Tbps ($\lambda_{\text{req}} = 10$), with an arrival rate of 1/8 and a duration of 20 minutes.

Fig. 2 presents a comparative evaluation of static and dynamic transceiver allocation strategies under AI-driven workload patterns, considering both without and with job queuing. The workloads are composed of TP, DP, PP, MoE and checkpoint with heterogeneous topological demands ranging from full-mesh to linear-chain model. In the absence of job queuing (Fig. 2(a)), the static allocation strategy exhibits significantly higher blocking probability due to dedicated allocation of pluggable transceivers across links. This leads to resource fragmentation, where available capacity cannot be effectively utilized across varying topology requirements. In contrast, the dynamic strategy substantially reduces blocking probability, approaching zero blocking as the number of pluggables reaches 360. Furthermore, dynamic allocation achieves better resource utilization across links, as evidenced by tighter min-max ranges, while the static case shows higher utilization peaks. On the other hand, when job queuing is enabled (Fig. 2(c)), blocking is effectively eliminated (due to an unbounded buffer) at the cost of scheduling delay. Under this setting, the static approach incurs significantly higher delays, particularly at lower pluggable counts, due to resource fragmentation and inefficient allocation. The dynamic strategy, combined with job queuing, rapidly reduces scheduling delay and achieves near-zero waiting time as the set of pluggables is 280. Fig. 2(b) and (d) illustrate the per-link pluggable usage, showing the average along with the minimum and maximum range. These plots capture how many pluggables are utilized on links participating in full-mesh communication (Jobs A and C), as well as in combined full-mesh and linear-chain communication patterns (Jobs A, B, and C). Dynamic allocation achieves more resource utilization across links, as evidenced by min-max ranges, while the static case shows linear utilization of pluggables. Notably, job queuing has zero blocking for AI traffic (Jobs B and C) and efficient pluggable utilization per link as compared to without job queuing. Overall, the resource savings are mainly from the dynamic transceiver pooling architecture.

Conclusions

We proposed an LLM-aware dynamic optical pluggable allocation method for multi-cluster DCI, considering queue-aware scheduling for heterogeneous AI training workloads. As a result, it eliminates job blocking and significantly reduces scheduling delay.

Acknowledgements

This work was supported by the project SHINE (PID2024-159781OB-I00) funded by MCIU/AEI /10.13039/501100011033 and by ERDF, EU and Horizon Europe PROTEUS-6G (101139134).

References

- [1] Z. Chen, S. Wang, Z. Tan, X. Fu, Z. Lei, P. Wang, et al., *A survey of scaling in large language model reasoning*, <https://arxiv.org/abs/2504.02181>, arXiv preprint arXiv:2504.02181, 2025.
- [2] A. Payong and S. Mukherjee, "Data parallelism: From basics to advanced distributed training", *DigitalOcean Community*, 2025. [Online]. Available: <https://www.digitalocean.com/community/conceptual-articles/data-parallelism-distributed-training>.
- [3] S. Ben Yochana, C. Avin, and G. Scalosub, "T3p: Topology-tailored tensor parallelism", in *Proceedings of the 2nd Workshop on Networks for AI Computing*, 2025.
- [4] Y. Huang et al., "Gpipe: Efficient training of giant neural networks using pipeline parallelism", in *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc., 2019.
- [5] S. Smith et al., *Using deepspeed and megatron to train megatron-turing nlg 530b, a large-scale generative language model*, <https://arxiv.org/abs/2201.11990>, arXiv preprint arXiv:2201.11990, 2022.
- [6] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity", *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022. [Online]. Available: <https://www.jmlr.org/papers/v23/21-0998.html>.
- [7] A. Q. Jiang et al., *Mixtral of experts*, <https://arxiv.org/abs/2401.04088>, Mistral AI, 2024.
- [8] Databricks, "Introducing dbrx: A new state-of-the-art open llm", *Databricks Blog*, 2024. [Online]. Available: <https://www.databricks.com/blog/introducing-dbrx-new-state-art-open-llm>.
- [9] Google Cloud, "Using multi-tier checkpointing for large ai training jobs", *Google Cloud Blog*, 2024. [Online]. Available: <https://cloud.google.com/blog/products/ai-machine-learning/using-multi-tier-checkpointing-for-large-ai-training-jobs>.
- [10] Z. Hong et al., "Scale matters: Large language models with billions (rather than millions) of parameters better match neural representations of natural language", *bioRxiv*, 2024.
- [11] DriveNets, *Hybrid scaling: Scale-up, scale-out, and scale-across gain new meaning*, Accessed: 2026-04-20, 2024. [Online]. Available: <https://drivenets.com/blog/hybrid-scaling-scale-up-scale-out-and-scale-across-gain-new-meaning/>.
- [12] Meta, *Scaling ai network infrastructure for large language model training at 100k+ gpu scale*, Accessed: 2026-04-20, 2025. [Online]. Available: <https://www.zenml.io/llmops-database/scaling-ai-network-infrastructure-for-large-language-model-training-at-100k-gpu-scale>.
- [13] Ciena, *What does dci really mean today? from scale across to submarine*, <https://www.ciena.com/insights/blog/2026/what-does-dci-really-mean-today-from-scale-across-to-submarine>, Accessed: 2026-04-20, 2026.
- [14] Nebius, "The difference between ai training and inference", *Nebius Blog*, 2025. [Online]. Available: <https://nebius.com/blog/posts/difference-between-ai-training-and-inference>.
- [15] D. Narayanan et al., "Efficient large-scale language model training on gpu clusters using megatron-lm", in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021. DOI: 10.1145/3458817.3476209.
- [16] C. Xie and B. Zhang, "Scaling optical interconnects for hyperscale data center networks", *Proceedings of the IEEE*, vol. 110, no. 11, pp. 1699–1713, Nov. 2022. DOI: 10.1109/JPRDC.2022.3178977.
- [17] Nokia, "800g zr/zr+: The new currency in ai-scale connectivity", *Nokia Blog*, 2024. [Online]. Available: <https://www.nokia.com/blog/800g-zr-zr+-the-new-currency-in-ai-scale-connectivity/>.